

SORTNET

Skrevet af

Simon Holm, Johannes Hesselager Rothe & Anne Sofie Nielsen Høj

Fasekoordinator: Johannes Hesselager Rothe

Indholdsfortegnelse

Introduktion.....	2
Programopbygning.....	2
<i>Programmeringsparadigmer</i>	3
Hjælpefunktioner	3
Programmer.....	4
<i>Comparator.py</i>	4
<i>Dataklasse – comparator</i>	4
<i>Make_comparator</i>	4
<i>Min_channel og max_channel</i>	4
<i>Is_standard</i>	4
<i>Apply</i>	5
<i>Subsets_of_two</i>	5
<i>All_comparators</i>	5
<i>Std_subsets_of_two</i>	6
<i>Std_comparators</i>	6
<i>To_program</i>	7
<i>Network.py</i>	8
<i>Dataclass</i>	8
<i>Empty_network</i>	8
<i>Append</i>	8
<i>Size</i>	8
<i>Max_channel</i>	9
<i>Is_standard</i>	9
<i>Apply</i>	9
<i>Outputs</i>	9
<i>Num_to_bi</i>	10
<i>All_outputs</i>	10
<i>Is_sorting</i>	10
<i>To_program</i>	11
Test og fejlfinding.....	11
Program (bilag).....	12
<i>Comparator.py</i>	12
<i>Network.py</i>	15

Introduktion

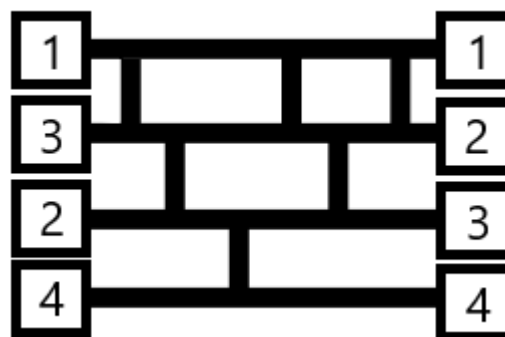
Dette projekt omhandler første fase af et større programmeringsprojekt. Projektet er et afsluttende gruppeprojekt i faget introduktion til programmering. I projektet anvendes Python 3.0 som programmeringssprog.

I dette projekt ligger fokuset på at udvikle funktioner, dataklasser og en struktur, der sammen skaber et sorteringsnetværk.

Et sorteringsnetværk er en klasse af sorteringsalgoritmer, der ved hjælp af comparators, sortere en række elementer. Ved hvert trin i et sorteringsnetværk sammenlignes 2 elementer ved brug af en comparator.

En comparator er en simpel funktion der sammenligner to værdier og bytter dem, hvis de ikke står i den ønskede rækkefølge.

I et typisk sorteringsnetværk bruges comparators til at arrangere elementer, i f.eks. en liste fra mindst til størst, som vist på Figur 1.



Figur 1: Eksempel på sorteringsnetværk med 4 kanaler

Programopbygning

Da det endelige program indeholder 21 forskellige funktioner, ville et samlet dokument være svært at navigere i, og derfor sværere at bruge. Derfor er disse funktioner delt op i to grupper;

- *Comparator*
- *Network*

Comparator-delen indeholder alle de funktioner der udnytter den "dataclass" comparator ligger i. Det kunne f.eks. være;

- *make_comparator*
- *apply*

Network indeholder alle de funktioner som anvendes igennem netværket. Det kunne f.eks. være;

- *size*
- *is_sorted*

Programmeringsparadigmer

Der er brugt flere forskellige programmeringsparadigmer, men med et hovedfokus på funktionel programmering. Projektet har valgt at fokusere på funktionel programmering, da opgavebeskrivelsen dikterer at programmets struktur skal inddeles i funktioner, derfor var det et naturligt valg at bibeholde denne struktur så vidt muligt.

Funktionel programmering egner sig ikke lige godt til alle programmeringsproblemer, og nogle af disse problemer, gav det bedre mening at bruge en imperativ programmerings løsning, f.eks. en *while*-løkke.

While-løkker har været en meget effektiv måde for denne gruppe at genbruge funktioner (ofte dem der involverer at hver comparator i et netværk skal gøre noget). Dette er gjort da gruppen har større kendskab til anvendelse af *while*-løkker i forhold til funktioner som *map*, *reduce* og *filter*. Disse funktioner er stadig blevet brugt, men deres brug er ikke maksimeret i forhold til alle situationer hvor de kunne have været anvendt.

Rekursiv programmering har ikke haft den store indflydelse i dette projekt, da gruppen i langt de fleste tilfælde vurderede at andre løsninger var at foretrække, enten baseret på let læselighed eller effektivitet.

Hjælpefunktioner

Gruppen havde diskuteret anvendelsen af hjælpefunktioner i starten, og blev enige om at tage det op, hvis en situation skulle opstå hvor et gruppemedlem vurderede at en funktion var for lang. Derefter ville det blive drøftet internt i gruppen og der ville blive taget en beslutning. Der kunne også skabes hjælpefunktioner hvis gruppen vurderede at en funktion ville fremme forståelsen og strukturen af programmet. Dette er f.eks. tilfældet for funktionen *num_to_bi*. Alle hjælpefunktioner vil blive gennemgået i rapporten.

Programmer

Programmets funktioner er opdelt i 2 Python filer, *comparator.py* og *network.py*.

Comparator.py

Dataklasse – comparator

Comparator.py bruger en dataclass, som gruppen har valgt at kalde "comparator". Denne dataclass bruger to attributter, *i* og *j*. *i* og *j* repræsenterer kanalerne som comparatoren er angivet på.

i og *j* har datatypen *int*, da de skal repræsentere kanaler, som altid er heltal.

Make_comparator

make_comparator funktionen returnerer en comparator med kanalerne *i* og *j*.

Min_channel og *max_channel*

Min_channel funktionen skal returnere den mindste kanal i comparatoren, hvorimod funktion *max_channel* funktionen returnerer den største kanal i en comparator.

Is_standard

Is_standard funktionen returnerer en bool værdi, som angiver om en given comparator er standard. Her forstås standard som at den sortere de mindste element, først i listen. Da funktionen *apply* sørger for at den mindste værdi på de to kanaler *i* og *j*, ender på *i*, ville en comparator dermed være standard hvis kanalen *i* har en mindre numerisk værdi end kanalen *j*.

F.eks.:

Hvis en comparator *c*, har kanalerne *i* = 1 og *j* = 0, vil den ikke være standard da *c.i* værdien er numerisk højere end *c.j* værdien.

Apply

apply funktionen anvender en comparator på en given liste. Funktionen har 2 input, en comparator og en liste.

Først laves en kopi af listen, dernæst anvendes en *if* statement, for at afgøre om de to værdier, som *comparatoren* peger på, skal byttes. Hvis de skal byttes, bliver der byttet rundt på dem i den nye liste, hvorefter den nye liste returneres. Hvis de ikke skal byttes, returneres den nye liste uden ændring.

Subsets_of_two

Subsets_of_two funktionen er en hjælpefunktion til funktionen *all_comparators*. Ved brug af *subsets_of_two* funktionen, er det muligt at lave en liste af alle kombinationer som har en længden 2. Med heltal fra 0 op til den usorteret listes længde, kan alle mulige kombinationer til en *comparator* findes.

Dette gøres ved hjælp af 2 *while* funktioner, en for hvert index.

Funktionen starter med en tom liste (*result*) samt en returnvariabel og *i* som er kontrolvariablen. Den første *while*-løkke skal køre så længe kontrolvariablen ikke overstiger længden af den usorterede liste, da funktionen ellers vil lave unødvendige subsets til *comparatoren*. Dernæst laves endnu en kontrol variabel *j* af samme grund.

Den efterfølgende *while*-løkke skal på samme måde køre så længe *j* er in. Denne *while* laver alle kombinationer af $[i, j]$ når *j* øges med 1 hvert trin. Når *j* når den usorteret listes længde lægges der 1 til *i* og starter forfra. Først når *i* overskrider den usorteret listes længde returneres listen *result*.

All_comparators

Formålet med funktionen *all_comparators* er at returnerer alle de mulige comparators på *n* kanaler.

Variablen *v* sættes til alle de mulige *subset* af 2, på *n* kanaler. Derved indeholder *v* alle mulige kanaler som comparators kan anvende.

Funktionen *make_comparator* bruger 2 heltal som input og returnerer en comparator med de to input som kanaler. Da *v* er en liste, med alle mulige kombinationer på *n* kanaler, returnerer *all_comparators* en liste hvor *map* med funktionen *make_comparator* er blevet brugt på alle de lister.

Her laves først en liste med kaldenavn v , som i dette tilfælde referer til hjælpefunktionen *subsets_of_two* med en *range* på n , altså på en vilkårlig liste med en længde mellem 0 og $n - 1$.

Derefter bruges *map* funktion til at lægge en funktion som indeholder den originale *make_comparator* over listen v , hvor de forskellige x værdier i det indeks oprettes henholdsvis med enten 0 eller 1, så output vil være en liste med alle former af sets i listen, som vises ved eksempel i doctesten. Her bruges *list(map)*, så den returnerer en liste.

Std_subsets_of_two

Med same fremgangsmåde som funktionen *subsets_of_two*, returnerer *std_subsets_of_two* alle kombinationer af heltallene fra 0 til længden af den usorteret liste. Med det tilføjede krav at de kombinationer hvor det første tal (i) mindre end det andet (j) er gyldige.

Forskellen kommer i hvordan funktionen tæller j . j starter lige under længden af den usorteret liste og mindskes ved hvert trin. Det andet *while* udtryk stopper når $i = j$, derfor når j aldrig at blive $\leq i$. Det medfører at funktionen returnerer den samme liste som *subsets*, men uden alle dubleanter som $[1,1]$, samt alle hvor i er større end j som eksempelvis $[3,2]$.

Std_comparators

Funktionen *std_comparators* har modsat *all_comparators*, til formål at returnere alle comparators på n kanaler der er standard.

Her starter funktionen med det samme princip som den tidligere funktion *std_comparators*, hvor der først laves en liste v , som referer til *std_subsets_of_two* med en *range* på n , altså på en vilkårlig liste mellem 0 og $n - 1$. Differencen, kommer da man benytter en standard sets funktion, som har sorteret i listen, så alt output ville være sorteret og dermed er standard.

Den returnerer en liste igen, da der bruges *list(map)*, hvor *map* funktionen lægger funktionen med *make_comparator* hen over den givne liste. Præcis som *all_comparators*.

```
v = std_subsets_of_two(range(n))
return list(map(lambda x: make_comparator(v[x][0], v[x][1]),
range(0, len(v))))
```

Koden for *std_comparators*, bemærk at funktionen *std_subsets_of_two* bliver lagt i en liste så den ikke skal kaldes for mange gange

To_program

To_program returnerer en liste med Python kommandoer der simulerer en given *comparator*. Hvis der f.eks. var en *comparator* på kanalerne 0 og 1, skal *to_program* lave en sekvens af kommander der ville give det samme resultat som at *apply comparator*en på en vilkårlig liste. Hvert element bliver lavet med string concatenation, og bruger f' strings, da dette gør det mere letlæseligt.

```
l1 = l1 + [f'if {var}[{c.i}] > {var}[{c.j}]:']
l1 = l1 + [f'{aux} = {var}[{c.i}]]
l1 = l1 + [f'{var}[{(c.i)}] = {var}[{c.j}]]
l1 = l1 + [f'{var}[{c.j}] = {aux}']
```

Eksempler på f' strings.

Koden tager udgangspunkt i en tom liste, og tilføjer derefter de følgende 4 elementer til listen:

1. Det første element tager udgangspunkt i en *if*-statement, da *comparator*en kun skal benyttes, hvis værdien på kanal *i* er større end på kanal *j*. Da det første element på en streng, skrevet som en *if*-statement i python der spørger om elementet på plads *i*, i den givne liste, er større end elementet på plads *j*.
2. Det næste element udfører første skridt, ved at flytte rundt på de 2 elementer, under antagelsen at de ikke er sorteret, (da det ellers ville være unødvendigt) ved at sætte *aux* variablen lig med værdien på den første kanal.
3. Det tredje element sætter værdien på plads *i* lig med værdien på plads *j*. Da værdien på plads *i* er gemt i *aux*, kan den overskrive værdien på plads *i*, med værdien på plads *j*, uden værdien *i* går tabt.
4. Sidste element sætter værdien på plads *j* lig med værdien i *aux*.

Derved er der blevet returneret en liste med fire elementer, der hvis de bliver indsat i et Python program, sorterer de to elementer ligesom en *comparator*.

Network.py

Projektet indeholder funktioner i *comparator* filen, der skal bruges i *network* filen. Disse funktioner deler navne på kryds, så for at holde programmet læseligt og funktionelt, får funktionerne der bliver importeret fra *comparator* filen suffixen: *_comparator*

```
from comparator import (comparator,
                        max_channel as max_channel_comparator,
                        is_standard as is_standard_comparator,
                        apply as apply_comparator,
                        to_program as to_program_comparator)
```

Import af funktioner fra *comparator.py*

Dataclass

network.py bruger også en dataclass. Den er navngivet "*Network*". Dataclassen bruger kun en attribut, *list*, *v* Som indeholder comparators.

Da *v* skal indeholde comparators, har gruppen valgt at lave *v* til en liste af comparators. Da hver comparator indeholder 2 kanaler, kunne et alternativ godt være en liste med tuples, men gruppen vurderede, primært ud fra størst kendskab til lister og attributter i dataclasses, at gøre det med denne struktur.

Empty_network

empty_network funktionens formål er at lave et netværk. Til at starte med vil netværket være en tom liste som der senere kan ligge comparators i.

Append

append funktionen bruges til at lægge comparators ind et netværk. Derefter returneres netværket igen, nu med en ny comparator.

```
net.v = net.v + [c]
return net
```

Det er muligt at anvende *.append* funktionen. Det undgås dog da gruppen har bedre forståelse af den brugte metode frem for *.append*.

Size

size funktionen returnerer antallet af comparators i et netværk. Siden et netværk kun kan bestå af comparators skal antallet af elementer i listen tælles. Denne funktion bruger *reduce* funktionen fra *functools*. Med *reduce* funktionen lægges 1 til *x* for hver *y*. Da *y* svarer til en plads i netværket tæller *x* antallet af elementer i netværket.

Når der bruges *reduce* fra *functools* er det vigtigt at huske at importere den i filen.

Max_channel

max_channel returnerer den kanal i alle netværkets comparators med den højeste numeriske værdi. Da funktionen *max_channel* i comparatoren delen allerede kan returnere den højeste numeriske værdi af de to kanaler i en comparator, er det ideelt at bygge denne funktion oven på den.

Reduce fra *functools* er en effektiv løsning til dette problem. *Reduce* bruges til at gå listen af comparators i netværket igennem, og hvis den højeste kanal på den *comparator*, er højere end det *reduce* kanalen allerede har i sin akkumulator. Bliver dette, det nye element i akkumulatoren, hvis ikke, beholdes det tidligere element. Når *reduce* har gået igennem listen, vil den derfor returnere den kanal med højst numeriske værdi.

Is_standard

is_standard funktionen tjekker om netværket kun indeholder standard comparators. Funktionen består af en *while*-løkke med 2 kontrolvariabler, *i* og *result*. Variablen *i* skal sørge for at *while*-løkken kun kører så længe der stadig er comparators i netværket. *Result* variabelens default værdi er sat til *True*.

Funktionen tjekker vha. *is_standard_comparator* om en comparator i netværket er standard. Hvis *is_standard_comparator* opdager en comparator der ikke er standard opdaterer den *result* til *False*. Til sidst returnerer funktionen *result*.

Apply

apply anvender, ligesom *apply_comparator*, en comparator på en liste. Dog i denne funktion bruges der et *while*-løkke der anvender *apply_comparator* på en liste. Her bruger man en kontrol variable der sikrer, at funktionen stopper med at anvende comparators når den har brugt alle comparators i netværket.

Outputs

Outputs funktionen bruges for at sætte netværket på alle inputs i *w*, og derefter returnerer den en liste uden dubletter.

Funktionens input er en liste som indeholder en liste med usorteret heltal.

While funktion bruges til at køre en løkke igennem listen, for at tjekke den for dubletter. Dette gøres ved at anvende *apply_all* funktionen på netværket, som dermed også anvender *comparatoren* på netværket. Hvorefter kontrolvariablen øges med 1, da den kun skal køre igennem listen en gang, eftersom *while* kun kører så længe *i* er mindre end længden af *w*

Hvorefter filter funktionen bliver lagt ovenpå listen w , da filter funktion fjerner de dupletter der findes. Efter dette bliver de fundne dubletter lagt i en anden list, og dermed fjernede fra w , hvorefter dem, der ikke er dupletter, bliver i w .

Num_to_bi

Funktionen *num_to_bi* konverterer et heltal n , til en liste, der repræsenterer tallet i binær form. *num_to_bi* funktionen bruges i de næste funktioner, hvor man laver en liste med binære tal.

Når man omskriver et heltal til binær form, finder man resten efter heltals division med tallet n og 2. Hvorefter funktionen kører rekursivt indtil man ikke kan dele tallet n mere

All_outputs

All_outputs bruges til at finde alle de mulige binære output med en vilkårlig længde fra netværket.

Den starter med at definere både i og j , kontrolvariablerne, hvor de begge bliver sat til 0. Da de skal starte fra enden af, derefter kører listen igennem. Indtil de i *while* løkken bliver større end n^2 . Denne *while* løkke kører så igennem listen, hvor den bliver defineret som *binarylist*. *Map* funktionen bruges derefter, til at lægge den tidligere defineret funktion *num_to_bi* oven på listen, så de bliver lavet om til deres tilsvarende binære kode. Derudover bliver der lagt en range ovenpå som laver en grænse. Som referer tilbage til kontrolvariablens grænse.

Efter denne grænse overskrides, vil funktion ende løkken og dermed køre videre. Hvorefter det samme udføres den anden kontrol variabel j .

Is_sorting

Is_sorting returner en bool værdi, der repræsenterer om det givne sorteringsnetværk ville sortere en liste med n heltal uanset placeringen af dem. Det gøres ved at skabe alle mulige kombinationer af binære output med længden n . Hvis sorteringsnetværket kan sortere alle disse lister, ville den derfor kunne sortere hvilken som helst liste af heltal.

Funktionen laver en variabel af funktionen *all_outputs*, så den ikke skal kalde funktionen hver gang og en bool variabel, *is_sorted*, der holder styr på om listen er sorteret. Ved at *is_sorted* er sat til true, kan vi bygge funktionen op således, at vi kun sætter den til false den ikke er sorteret, men hvis den når igennem alle lister uden fejl, så returnere den *is_sorted*.

Ved brug af en nested *while* løkke, bliver alle elementer i hver liste gennemgået. Funktionen starter med at sammenligne om hvert element i liste *i* er mindre end det næste element. Hvis det er, bliver næste element sammenlignet og dermed forsætter løkken indtil der ikke er flere elementer i den sub liste. Hvis et element ikke er mindre end det næste element, bliver *is_sorted* sat lig med *false*.

Når en sub liste er tjekkes færdig, bliver der også tjekket om *is_sorted* er *true*. Hvis den er *true*, altså at listen indtil videre er sorteret, går den videre til næste sub liste og går igennem den. Hvis den ikke er *true*, kan der allerede konkluderes at sorteringsnetværket ikke sortere alting, og dermed giver det ikke mening at tjekke resten af listen, men der kan bare returneres *is_sorted*, som er *false*.

To_program

To_program funktionen returnerer en sekvens af instruktioner, i form af en liste, til et program, der simulerer netværket i Python. *To program* bruger en *while*-løkke der, så længe der stadig er comparators i netværket, fylder en liste med instrukser vha. *to_program_comparator* funktionen.

```
l1 = []
i = 0
while i < len(net.v):
    l1 = l1 + to_program_comparator(net.v[i], var, aux)
    i = i + 1
return(l1)
```

Bemærk den bevidste *while* – løkke struktur

Test og fejlfinding

Under projektet har gruppen løbende testet funktionerne individuelt og sammen. Det er vigtigt at teste programmet så grundigt som muligt. Det gøres ved at teste funktionerne med inputs som ikke er meningen til programmet. Det kan f.eks. være negative værdier. Det gøres for at finde evt. fejl i funktionen som ikke var opstået ellers.

Program (bilag)

Comparator.py

```
from dataclasses import dataclass
@dataclass
class comparator:
    i: int
    j: int

def make_comparator(i, j) -> comparator:
    """Makes a comparator. i and j, needs to be >=0.
    >>>make_comparator(4,5)
    comparator(4,5)
    """
    return comparator(i,j)

def min_channel(c:comparator) -> int:
    """Returns the channel that contains the smallest int.
    >>>min_channel(comparator(5,3))
    3
    """
    if c.i < c.j:
        return c.i
    else:
        return c.j

def max_channel(c:comparator) -> int:
    """Returns the channel that contains the biggest int.
    >>>max_channel(comparator(2,4))
    4
    """
    if c.i > c.j:
        return c.i
    else:
        return c.j

def is_standard(c:comparator) -> bool:
    """Determines whether a comparator is standard or not.
    >>>is_standard(comparator(2,1))
    false
    """
    return c.i < c.j
```

```

def apply(c:comparator, w:list[int]) -> list[int]:
    """Applies comparator to a list of intergers.
    >>>apply(comparator(0,1), [3,2,1,4])
    [2,3,1,4]
    """
    new_list = w[:]
    if w[c.i] > w[c.j]:
        new_list[c.i], new_list[c.j] = w[c.j], w[c.i]
    return new_list

def subsets_of_two(v) -> list:
    """Returns an index-list of all subsets of a list, with the length of 2
    >>>subsets_of_two([1,2,3])
    [[0,1],[0,2],[1,0],[1,2],[2,0],[2,1]]
    """
    result = []
    i = 0
    while i < len(v):
        j = 0
        while j < len(v):
            if i != j:
                result.append([i,j])
            j = j + 1
        i = i + 1
    return result

def all_comparators(n:int,) -> list[comparator]:
    """Returns all possible comparators on n channels. n needs to be >=1.
    >>>all_comparators(3)
    [comparator(0,1),comparator(0,2),comparator(1,0),
    comparator(1,2),comparator(2,0),comparator(2,1)]
    """
    v = subsets_of_two(range(n))
    return list(map(lambda x: make_comparator(v[x][0], v[x][1]),
        range(0,len(v))))

```

```

def std_subsets_of_two(v:list) -> list:
    """Returns a index-list of all subsets of a list, with the length of 2,
    that fits as inputs for a standard comparator.
    >>>std_subsets_of_two([1,2,3])
    [[0,2],[0,1],[1,2]]
    """
    result = []
    i = 0
    while i < len(v):
        j = len(v)-1
        while j > i:
            if i != j:
                result.append([i,j])
            j = j - 1
        i = i + 1
    return result

def std_comparators(n:int) -> list[comparator]:
    """Returns all standard comparators on n channels. n needs to be >=1.
    >>>std_comparators(3)
    [comparator(0,2),comparator(0,1),comparator(1,2)]
    """
    v = std_subsets_of_two(range(n))
    return list(map(lambda x: make_comparator(v[x][0], v[x][1]),
        range(0,len(v))))

def to_program(c:comparator, var:str, aux:str) -> list[str]:
    """Returns a sequence of instructions for simulating the
    comparator in Python.
    >>>to_programc(comparator(0,1),w1,aux)
    ['if w1[0] > w1[1]:', 'aux = w1[0]', 'w1[0] = w1[1]', 'w1[1] = aux']
    """
    l1 = []
    l1 = l1 + [f'if {var}[{c.i}] > {var}[{c.j}]:']
    l1 = l1 + [f'{aux} = {var}[{c.i}]']
    l1 = l1 + [f'{var}[{c.i}] = {var}[{c.j}]']
    l1 = l1 + [f'{var}[{c.j}] = {aux}']
    return l1

```

Network.py

```
from functools import reduce

from dataclasses import dataclass
from comparator import (comparator,
                        max_channel as max_channel_comparator,
                        is_standard as is_standard_comparator,
                        apply as apply_comparator,
                        to_program as to_program_comparator)

@dataclass
class Network:
    v: list[comparator]

def empty_network() -> Network:
    """Makes an empty network.
    >>>empty_network()
    v[]
    """
    return Network([])

def append(c:comparator, net:Network) -> Network:
    """Adds a comparator at the end of a network.
    >>>append(comparator(3,4), v[])
    v[comparator(3,4)]
    """
    net.v = net.v + [c]
    return net

def size(net:Network) -> int:
    """Returns the number of comparators in net.
    size(net.v[comparator(3,4),comparator(4,5)])
    2
    """
    return reduce(lambda x,y: x+1,net.v,0)

def max_channel(net:Network) -> int:
    """Returns the highest channel."""
    return (reduce(lambda x, comparator: max_channel_comparator(comparator)
                    if max_channel_comparator(comparator) > x else x,
                    net.v,0))
```



```

def is_standard(net:Network) -> bool:
    """Checks if net only contains standard comparators.
    >>>is_all_standard([comparator(0,1), comparator(1,2), comparator(0,2)])
    True
    """
    i = 0
    result = True
    while i < size(net):
        if is_standard_comparator(net.v[i]) == False:
            result = False
        i = i+1
    return result

def apply(net:Network, w:list[int]) -> list[int]:
    """Applies comparatornetwork to a list of intergers.
    >>>apply([comparator(0,1), comparator(1,2)], [2,1,3])
    [1,2,3]
    """
    i = 0
    while i < size(net):
        w = apply_comparator(net.v[i],w)
        i = i + 1
    return w

def outputs(net:Network, w:list[list[int]]) -> list[list[int]]:
    """Applies net on all inputs in w and returns a list with no duplicates.
    >>>outputs([comparator(0,1), comparator(1,2), comparator(0,2)],
    [[2,1,3], [1,3,2], [1,3]])
    [1,2,3], [1,3]
    """
    i = 0
    while i < len(w):
        w[i] = apply(net,w[i])
        i = i + 1
    duplikat = []
    w = list(filter(lambda x: (x not in duplikat)
                        and not duplikat.append(x), w))
    return w

def num_to_bi(n:int) -> list:
    """Converts a number to its corresponding binary code. n needs to be >=0.
    >>>num_to_bi(5)
    [1, 0, 1]
    """
    if n > 1:
        return num_to_bi(n//2) + [n%2]
    else:

```

```

        return [n%2]
def all_outputs(net:Network, n: int) -> list[list[int]]:
    """Returns all posible binary outputs of length n from net."""
    i = 0
    j = 0
    while i < 2**n:
        binarylist = list(map(lambda x: num_to_bi(x),list(range(0,2**n))))
        i = i+1
    while j < 2**n:
        binarylist[j] = [0] * (n-len(binarylist[j])) + binarylist[j]
        j = j + 1
    return outputs(net,binarylist)

def is_sorting(net: Network, size:int) -> bool:
    """Checks if net is a sortingnetwork for n input.
    >>>is_sorting([comparator(0,1), comparator(i=0, j=2),
    comparator(i=1, j=2),],[[0, 0, 0], [1, 0, 1], [1, 1, 1]])
    False
    """
    is_sorted = True
    i = 0
    all_combinations = all_outputs(net, size)
    while i < len(all_combinations):
        j = 0
        while j < len(all_combinations[i])-1:
            if all_combinations[i][j+1] < all_combinations[i][j]:
                is_sorted = False
            j = j+1
        if is_sorted == True:
            i = i + 1
        else:
            return is_sorted
    return is_sorted

def to_program(net:Network, var:str, aux:str) -> list[str]:
    """Returns a sequence of instruction for simulating the
    network in python.
    """
    l1 = []
    i = 0
    while i < len(net.v):
        l1 = l1 + to_program_comparator(net.v[i],var,aux)
        i = i + 1
    return(l1)

```